

1. The Database Consolidation (1 Container)

Instead of 3 separate database containers, you will run one **PostgreSQL** instance with three **logical databases** or three **schemas**.

- **The Move:** In your docker-compose.yml, you only need one db service.
- **The Setup:** Use a initialization script (init.sql) in /docker-entrypoint-initdb.d/ to create the three databases on startup, or simply move all tables into one database using different schemas:
 - identity_schema (Your RBAC/Tenant tables)
 - agent_platform_schema
 - execution_engine_schema

2. The Unified Backend (1 Container)

This is where the "All APIs in one layer" requirement happens. You will use **FastAPI APIRouter** to mount the other teams' work into your main application.

- **Step A: Structural Merge** Move the code from the other two backends into sub-packages of your main project:

Plaintext

/backend

└─ main.py

└─ app/

| └─ identity/ # Your Module

| └─ agents/ # Team 2

| └─ execution/ # Team 3

- **Step B: The Master Router (main.py)** In your main FastAPI entry point, import and include their routers with prefixes:

Python

```
from app.identity.api import router as identity_router
```

```
from app.agents.api import router as agents_router
```

```
app = FastAPI()
```

```
app.include_router(identity_router, prefix="/auth", tags=["Identity"])
```

```
app.include_router(agents_router, prefix="/agents", tags=["Agents"])
```

- **Step C: RBAC Integration** Since your module is **Identity and RBAC**, your middleware/dependencies will now sit "above" their routes. You can now easily apply your tenant isolation logic to their endpoints in one place.

3. The Unified UI (1 Container)

Since both teams used **Next.js**, you need to merge the `pages/` or `app/` directories.

- **The Shell:** Pick one Next.js project to be the "Root."
- **Merging:** Copy the components and page files from the other frontend into the Root project.
- **The `next.config.js`:** Ensure all environment variables (API URLs) point to the single Backend container.

4. The AI Agents (1 Container)

This container should stay isolated as requested. It will likely run the heavy logic, LangChain, or LLM orchestration that the "Agent Creation Platform" requires.

- **Communication:** The **Backend Container** will call this **AI Agent Container** via internal Docker networking (e.g., `http://ai-agents:8000`).

Updated `docker-compose.yml` Structure

This is what your final configuration should look like to meet the "4 container" rule:

YAML

services:

db:

image: postgres:15

environment:

- POSTGRES_DB=main_db
- POSTGRES_USER=admin
- POSTGRES_PASSWORD=password

backend:

build: ./backend

depends_on:

- db

environment:

- DATABASE_URL=postgresql://admin:password@db:5432/main_db
- AGENT_SERVICE_URL=http://ai-agents:8080

ui:

build: ./frontend

ports:

- "3000:3000"

environment:

- NEXT_PUBLIC_API_URL=http://localhost:8000

ai-agents:

build: ./ai-agent-module

This keeps the agent logic separate as requested

Immediate Next Steps

1. **Dependency Audit:** Compare the requirements.txt of all 3 backends. If Team A uses SQLAlchemy 1.4 and Team B uses SQLAlchemy 2.0, you need to pick one and fix the breaking changes now.
2. **Environment Variables:** Create a single .env file that contains the secrets for all three modules.
3. **Namespace Check:** Ensure no two teams named their FastAPI startup variable app. Rename them to identity_app, agent_app, etc., before importing them into the final main.py.